CITY CARE

Documentación completa 3.2 Almacenamiento de evidencias en MinIO

Integración del endpoint de reportes con almacenamiento de objetos

La etapa 3.2 sustituye la conservación local provisional por almacenamiento privado en MinIO. El backend sube la fotografía, registra su referencia en MySQL, elimina el archivo temporal y entrega una URL firmada que la aplicación Android puede consultar durante un tiempo limitado.

Indicador	Definición
Estado	Completo y validado
Contenedor	minio/minio en Docker
Bucket	city-care privado
API interna	http://127.0.0.1:9000
Acceso desde emulador	http://10.0.2.2:9000
Vigencia de URL	900 segundos
Suite funcional	52 de 52 pruebas aprobadas

Resultado de la etapa

El endpoint POST /api/v1/reports conserva su contrato multipart y ahora finaliza la evidencia en MinIO. Una respuesta 201 incluye fotoUrl y estadoEvidencia Disponible. No se agregó una migración porque la tabla report_evidences ya contenía provider, status, bucket y object_key desde la etapa 3.1.

1 Objetivo y alcance

- Conectar el backend AdonisJS con el servicio MinIO que se ejecuta en Docker.
- Mantener el bucket privado y evitar credenciales dentro de la aplicación móvil.
- Transferir cada fotografía validada al bucket city-care.
- Guardar en MySQL únicamente la metadata y la clave lógica del objeto.
- Eliminar el archivo local de preparación después de la operación.
- Compensar el objeto remoto si la transacción de base de datos falla.
- Generar una URL firmada y temporal compatible con Android Emulator.
- Documentar el error de indisponibilidad en OpenAPI y regenerar Orval.
- Validar la integración con pruebas automatizadas y una operación real de MinIO.

2 Arquitectura

Componente	Dirección o recurso	Responsabilidad
Flutter Android	10.0.2.2:3333	Envía el reporte multipart y recibe fotoUrl.
AdonisJS	Puerto 3333	Valida, coordina MinIO y persiste en MySQL.
MinIO API	127.0.0.1:9000	Recibe objetos desde el backend.
MinIO Console	127.0.0.1:9001	Administración manual del servicio y bucket.
Bucket	city-care	Conserva las fotografías de forma privada.
MySQL	report_evidences	Conserva bucket, object_key, provider y status.
OpenAPI y Orval	city-care.v1.yaml	Publican fotoUrl y el posible error 503.

Se utilizan dos endpoints lógicos para el mismo MinIO. El backend usa 127.0.0.1 porque corre en Windows; la URL entregada a Android se firma con 10.0.2.2, que es el alias mediante el cual el emulador alcanza a la computadora anfitriona.

3 Flujo completo de creación

1. Flutter obtiene previamente un token mediante el endpoint de inicio de sesión.
2. La aplicación construye multipart/form-data con categoría, ubicación, descripción y foto.
3. El middleware auth valida el token y obtiene al usuario; el formulario no puede elegir user_id.
4. VineJS valida tipos, rangos, fotografía, extensión y tamaño máximo de 10 MB.
5. El controlador verifica categoría activa, descripción condicional, coordenadas y origen de ubicación.
6. La fotografía se mueve a storage/report-evidence-staging con una clave UUID organizada por año y mes.
7. El servicio comprueba que el bucket city-care exista; si recibe 404 puede crearlo cuando las credenciales lo permiten.
8. El backend abre el archivo como stream y lo envía a MinIO mediante la API compatible con S3.
9. MinIO conserva Content-Type y el nombre original como metadata; no se usa el nombre del usuario como ruta.
10. Después de una subida correcta comienza la transacción de MySQL.
11. La transacción crea reports, genera el folio y crea report_evidences con provider minio y status stored.
12. Si MySQL confirma la transacción, el backend carga las relaciones del reporte.
13. El servicio genera fotoUrl firmada con duración de 900 segundos.
14. El bloque finally elimina el archivo de preparación local aunque la operación termine con error.
15. El backend responde HTTP 201 con estado Pendiente y estadoEvidencia Disponible.

4 Flujo resumido

Entrada	Procesamiento	Persistencia	Salida
Token, datos, GPS y foto	Validación, staging y subida S3	Objeto en MinIO y referencia en MySQL	201, folio, fotoUrl y Disponible

4.1 Puntos de control

Frontera	Comprobación
Flutter a backend	Token válido, multipart completo y fotografía aceptada.
Backend a MinIO	Bucket disponible, objeto almacenado y metadata recibida.
Backend a MySQL	Reporte y evidencia confirmados dentro de la transacción.
Backend a Flutter	fotoUrl firmada, estado Pendiente y evidencia Disponible.

5 Configuración por variables de entorno

Las credenciales reales permanecen únicamente en .env, archivo excluido de Git. .env.example documenta la estructura sin contener secretos.

Variable	Valor de desarrollo	Uso
MINIO_ENDPOINT	127.0.0.1	Host utilizado por el backend para subir y eliminar.
MINIO_PUBLIC_ENDPOINT	10.0.2.2	Host incluido en la URL firmada para Android Emulator.
MINIO_PORT	9000	Puerto de la API S3; no usar el 9001 de la consola.
MINIO_USE_SSL	false	HTTP local. En producción debe cambiar a true con TLS.
MINIO_ACCESS_KEY	Secreto	Identificador de la credencial del backend.
MINIO_SECRET_KEY	Secreto	Clave privada; nunca se devuelve a Flutter.
MINIO_BUCKET	city-care	Bucket privado de evidencias.
MINIO_REGION	us-east-1	Región usada para firmar peticiones S3.
MINIO_SIGNED_URL_EXPIRES_IN	900	Duración de fotoUrl en segundos.

start/env.ts valida la presencia y el tipo de cada variable antes de iniciar la aplicación. La clave secreta se maneja como Secret y se libera únicamente al construir el cliente S3.

6 Configuración de red

Origen	Destino correcto	Razón
Backend en Windows	127.0.0.1:9000	Docker publica la API de MinIO en el host.
Navegador de Windows	127.0.0.1:9001	Acceso a la consola administrativa.
Android Emulator	10.0.2.2:3333	Acceso al backend de la computadora.
fotoUrl en Android	10.0.2.2:9000	Acceso al objeto firmado de MinIO.

No debe utilizarse localhost dentro del emulador: allí localhost representa al propio dispositivo virtual. Tampoco debe usarse 10.0.2.2 como endpoint interno del backend, porque ese alias pertenece a la red del emulador.

7 Organización del objeto en MinIO

```
reports/AÑO/MES/UUID.extensión
```

Elemento	Ejemplo	Propósito
Prefijo	reports	Separa evidencias de otros recursos futuros.
Año	2026	Facilita organización y políticas de ciclo de vida.
Mes	11	Evita demasiados objetos bajo un único prefijo.
UUID	550e8400-e29b-...	Evita colisiones y exposición del nombre original.
Extensión	png	Conserva un formato coherente con el archivo validado.

El objeto conserva Content-Type y una versión codificada del nombre original como metadata. La ruta no contiene correo, nombre, ID de usuario ni dirección del reporte.

8 Registro en MySQL

Columna	Valor después de 3.2	Significado
report_id	ID del reporte	Relación con reports.id.
kind	image	Tipo de evidencia.
storage_provider	minio	El archivo final está en MinIO.
storage_status	stored	La transferencia se completó.
bucket	city-care	Bucket donde reside el objeto.
object_key	reports/2026/11/UUID.png	Clave exacta para consultar o eliminar.
original_name	evidencia.png	Nombre recibido del cliente.
mime_type	image/png	Tipo MIME validado.
size_bytes	Tamaño real	Cantidad de bytes de la fotografía.

MySQL no almacena los bytes de la fotografía ni una URL firmada. La URL es temporal y se genera a partir de bucket y object_key. Esto evita guardar enlaces vencidos.

9 URL firmada

- Autoriza temporalmente una lectura concreta sin hacer público el bucket.
- Incluye firma, credencial, fecha y tiempo de expiración en la cadena de consulta.
- Durante desarrollo usa 10.0.2.2:9000 para ser accesible desde Android Emulator.
- Vence después de 900 segundos; cambiar ese valor no requiere modificar el código.
- No debe almacenarse como dato permanente ni compartirse en registros o mensajes.
- Cuando exista consulta de reportes, el backend deberá generar una URL nueva en cada respuesta.

10 Respuesta del endpoint

```
{
  "id": 1,
  "folio": "CC-2026-000001",
  "categoria": "Bache",
  "latitud": 20.67665,
  "longitud": -103.34751,
  "origenUbicacion": "gps",
  "estado": "Pendiente",
  "fotoUrl": "http://10.0.2.2:9000/city-care/reports/...?...",
  "estadoEvidencia": "Disponible",
  "creadoEn": "2026-11-01T12:00:00.000Z"
}
```

fotoUrl es obligatoria en la respuesta 201 de creación. Si la evidencia no está almacenada correctamente, el backend no presenta el reporte como una creación completa.

11 Uso desde Flutter

Flutter no recibe MINIO_ACCESS_KEY ni MINIO_SECRET_KEY. Solo utiliza fotoUrl como cualquier URL HTTP. La URL puede mostrarse con Image.network mientras permanezca vigente.

```
final response = await enviarReporte(formData, token);
final fotoUrl = response.fotoUrl;
Image.network(fotoUrl);
```

12 Consistencia y compensaciones

Situación	Acción automática	Estado final
Validación falla	No se sube la fotografía.	Sin objeto y sin reporte.
MinIO no responde	Se devuelve 503 y se elimina staging.	Sin reporte en MySQL.
Subida correcta y MySQL falla	Se elimina el objeto remoto.	Sin objeto huérfano.
Subida y transacción correctas	Se elimina staging y se firma fotoUrl.	Objeto stored y reporte creado.
Limpieza remota falla	Se registra error estructurado.	Requiere revisión operativa.
Limpieza local falla	Se registra advertencia sin ocultar el reporte creado.	Objeto final disponible.

La transacción de MySQL no puede incluir físicamente a MinIO porque son sistemas distintos. Por ello se aplica una transacción con compensación: si falla la segunda parte, se revierte explícitamente la primera.

13 Manejo de errores

HTTP	Código	Descripción
401	E_UNAUTHORIZED_ACCESS	Token faltante, inválido o vencido.
422	E_VALIDATION_ERROR	Campo, coordenada o fotografía inválida.
422	E_REPORT_CATEGORY_INVALID	Categoría inexistente o inactiva.
422	E_REPORT_DESCRIPTION_REQUIRED	Otro sin descripción.
422	E_REPORT_LOCATION_INVALID	Ubicación 0,0.
503	E_REPORT_STORAGE_UNAVAILABLE	MinIO no responde o rechaza la operación.

14 Componentes implementados

Archivo	Responsabilidad
config/minio.ts	Centraliza endpoints, credenciales, bucket, región y expiración.
start/env.ts	Valida todas las variables MINIO antes del arranque.
minio_report_evidence_service.ts	Bucket, subida, eliminación y URL firmada mediante S3.
report_evidence_staging_service.ts	Preparación privada, UUID y eliminación local.
reports_controller.ts	Orquesta validación, MinIO, transacción, compensación y respuesta.
handler.ts	Transforma la indisponibilidad de almacenamiento en error JSON 503.
report_evidence.ts	Representa la metadata almacenada en MySQL.
city-care.v1.yaml	Declara fotoUrl obligatoria y respuesta 503.
city-care.ts	Cliente Orval actualizado para el frontend TypeScript.

15 Contrato OpenAPI y Orval

- POST /api/v1/reports continúa recibiendo multipart/form-data.
- ReportPublic declara fotoUrl como propiedad obligatoria.
- estadoEvidencia se devuelve como Disponible después de la subida.
- La respuesta 503 documenta E_REPORT_STORAGE_UNAVAILABLE.
- Orval regenera createReport sin cambiar la manera en que Flutter construye su petición.
- El YAML sigue siendo la fuente contractual y el SDK generado no debe editarse manualmente.

16 Dependencias

La implementación utiliza @aws-sdk/client-s3 y @aws-sdk/s3-request-presigner. MinIO implementa la API S3, por lo que este cliente mantiene compatibilidad y permite firmar URLs con un endpoint público distinto al usado para la subida.

El cliente oficial de MinIO evaluado inicialmente introducía vulnerabilidades transitivas de severidad alta. Se reemplazó antes del cierre. La auditoría final no atribuye vulnerabilidades altas a la integración de almacenamiento.

17 Pruebas y evidencia de validación

Comprobación	Resultado
Contenedor Docker	minio activo y puertos 9000 y 9001 publicados.
Bucket	city-care localizado correctamente.
Subida real	Objeto temporal de 37 bytes almacenado.
Consulta real	Tamaño y metadata consultados correctamente.
URL firmada	Host generado: 10.0.2.2:9000.
Limpieza real	Objeto de diagnóstico eliminado; no quedó basura.
Suite funcional	52 de 52 pruebas aprobadas.
TypeScript	Sin errores.
ESLint	Sin errores.
Build	Compilación de producción correcta.
OpenAPI	Contrato validado por Orval.
SDK	Cliente regenerado correctamente.

Las pruebas funcionales usan una implementación controlada durante NODE_ENV=test para conservar independencia de Docker. La conectividad de MinIO se comprobó además con una prueba real separada de subida, firma y eliminación.

18 Operación diaria

Paso	Acción
1	Iniciar Docker y confirmar que el contenedor minio esté activo.
2	Confirmar que el bucket city-care exista y permanezca privado.
3	Iniciar el backend con npm run dev.
4	Usar http://10.0.2.2:3333 desde Android Emulator.
5	Enviar el reporte multipart con token Bearer.
6	Confirmar HTTP 201, fotoUrl y estadoEvidencia Disponible.
7	Abrir fotoUrl desde el emulador antes de que venza.
8	Verificar en MinIO Console el objeto bajo reports/AÑO/MES.
9	Verificar en report_evidences los valores minio, stored y city-care.

19 Diagnóstico de problemas

Síntoma	Causa probable	Revisión
503 al crear reporte	MinIO apagado o credenciales incorrectas.	docker ps, endpoint, access key y secret key.
Bucket no encontrado	Nombre distinto o permiso insuficiente.	MINIO_BUCKET y permisos de creación o lectura.
fotoUrl no abre en Android	Se firmó con 127.0.0.1.	MINIO_PUBLIC_ENDPOINT debe ser 10.0.2.2.
Firma inválida	Host, región, reloj o credenciales no coinciden.	Endpoint público, región y hora del sistema.
URL vencida	Superó 900 segundos.	Solicitar al backend una URL nueva en la consulta futura.
Archivo no aparece	Falló la subida antes de MySQL.	Logs y E_REPORT_STORAGE_UNAVAILABLE.
Objeto sin fila MySQL	Falló la compensación.	Buscar log report_evidence_minio_compensation_failed.
Staging conserva archivo	Falló la limpieza local.	Buscar log report_evidence_staging_cleanup_failed.

20 Seguridad y producción

- Mantener el bucket privado y bloquear acceso anónimo.
- Crear una credencial exclusiva para City Care con permisos mínimos sobre el bucket.
- No usar credenciales root de MinIO en producción.
- Activar HTTPS y cambiar MINIO_USE_SSL a true.
- Configurar MINIO_PUBLIC_ENDPOINT con el dominio real, no con 10.0.2.2.

- Rotar credenciales y mantenerlas fuera de Git, documentación y registros.
- Definir copias de seguridad, retención y políticas de ciclo de vida.
- Monitorear errores 503, fallos de compensación y capacidad del bucket.
- Revisar periódicamente npm audit sin aplicar --force de manera automática.

21 Alcance posterior

La etapa 3.2 resuelve la subida y entrega inicial de la fotografía. Cuando se implementen los endpoints de consulta de reportes, cada respuesta deberá generar una fotoUrl nueva porque las URLs no se guardan y expiran. También podrán añadirse múltiples evidencias, miniaturas, procesamiento de imágenes, antivirus, CNN y políticas de retención.

22 Cierre de la etapa

La etapa 3.2 queda concluida: el backend valida la imagen, la almacena en el bucket privado city-care, registra su referencia en MySQL y entrega una URL firmada temporal. Si MySQL falla después de la subida, el objeto se elimina como compensación; si MinIO no responde, no se crea un reporte incompleto.

Las decisiones centrales son guardar referencias en lugar de binarios, no persistir URLs temporales, separar los endpoints interno y público para Android Emulator y mantener las credenciales exclusivamente en el backend. OpenAPI, Orval, la compilación y las 52 pruebas confirman el cierre técnico de esta etapa.